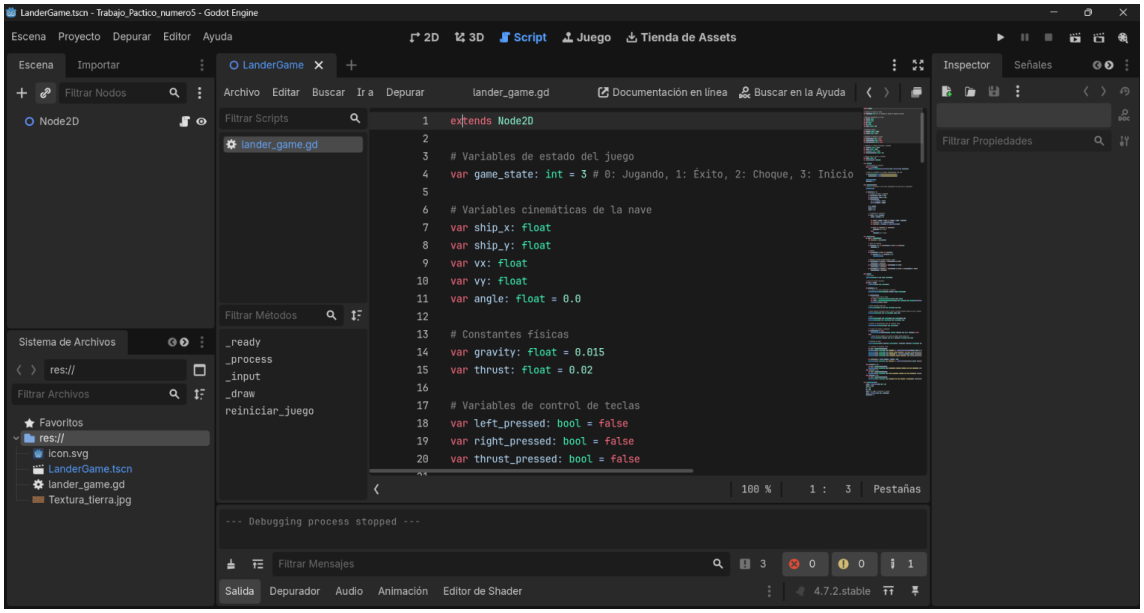
Garcia Sad Tomás Agustín
Legajo:14077

Trabajo Práctico N°5

Desarrollar el mismo ejercicio práctico del parcial sobre la herramienta desarrollo de videojuegos elegida: Godot

Código e implementación:




```gdscript
21
22  # Constantes de terreno, plataforma y colisión
23  var ground_y: float = 550.0
24  var pad_x: float = 350.0
25  var pad_width: float = 100.0
26  var max_safe_velocity: float = 1.5
27
28  # Arreglos para el fondo de estrellas
29  var stars: Array = []
30  var textura_tierra: Texture2D
31
32  func _ready():
33      # Inicialización de estrellas
34      for i in range(100):
35          stars.append(Vector2(randf_range(0, 800), randf_range(0, ground_y)))
36
37      # Carga de la textura. Si no existe, textura_tierra será null
38      if ResourceLoader.exists("res://Textura_tierra.jpg"):
```

```
39      textura_tierra = load("res://Textura_tierra.jpg")
40
41      reiniciar_juego()
42      game_state = 3
43
44  func _process(_delta):
45      # Fuerza el redibujado en cada frame (equivalente al bucle draw de Processing)
46      queue_redraw()
47
48      if game_state == 0:
49          # 1. Lógica de física y controles
50          if left_pressed: angle -= 0.05
51          if right_pressed: angle += 0.05
52          if thrust_pressed:
53              vx += sin(angle) * thrust
54              vy -= cos(angle) * thrust
55
56          vy += gravity
57          ship_x += vx
58          ship_y += vy
```

```
59
60          # 4. Detección de colisiones
61          if ship_y + 15 >= ground_y:
62              ship_y = ground_y - 15
63
64              var on_pad = ship_x >= pad_x and ship_x <= pad_x + pad_width
65              var safe_speed = vy < max_safe_velocity
66              var safe_angle = cos(angle) > cos(deg_to_rad(30))
67
68              if on_pad and safe_speed and safe_angle:
69                  game_state = 1 # Éxito
70              else:
71                  game_state = 2 # Choque
72
73  func _input(event):
74      if event is InputEventKey:
75          var is_pressed = event.pressed
76
77          # Inicio del simulador
78          if game_state == 3 and event.keycode == KEY_S and is_pressed:
79              game_state = 0
```

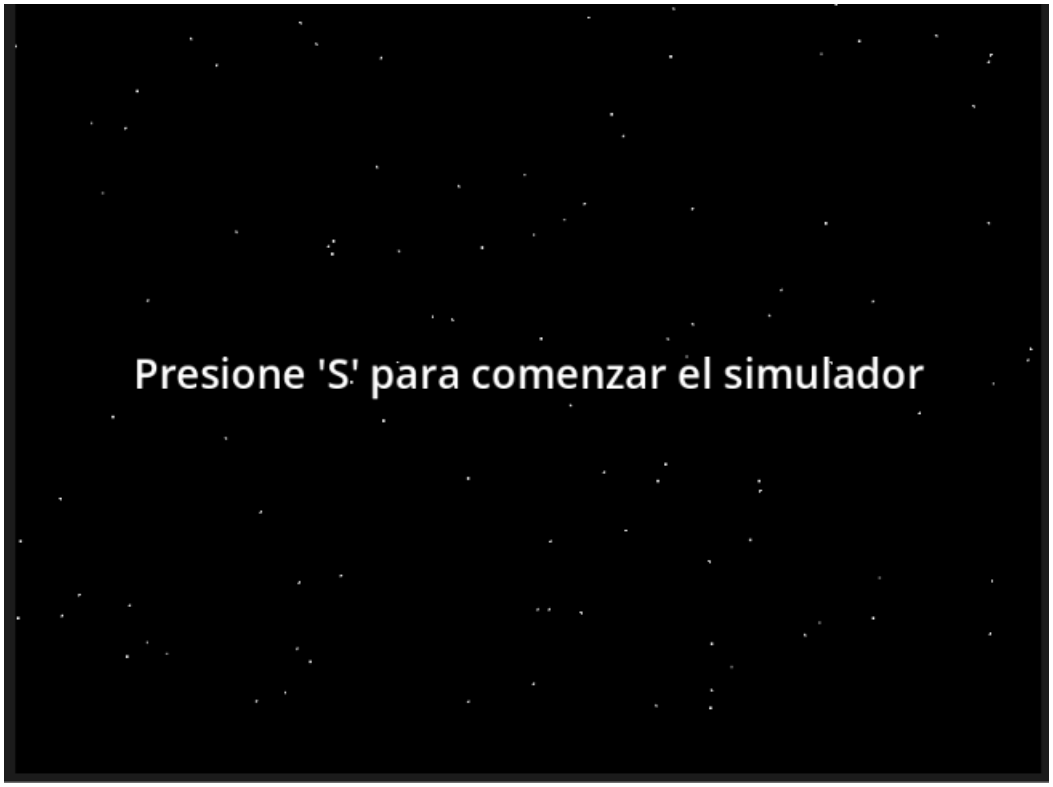
```
80
81          # Reinicio
82          if event.keycode == KEY_R and is_pressed:
83              if game_state == 1 or game_state == 2:
84                  reiniciar_juego()
85
86          # Controles de vuelo (incluye flechas y WASD)
87          if event.keycode == KEY_LEFT or event.keycode == KEY_A:
88              left_pressed = is_pressed
89          if event.keycode == KEY_RIGHT or event.keycode == KEY_D:
90              right_pressed = is_pressed
91          if event.keycode == KEY_SPACE or event.keycode == KEY_UP or event.keycode == KEY_W:
92              thrust_pressed = is_pressed
93
94  func _draw():
95      # Fondo negro
96      draw_rect(Rect2(0, 0, 800, 600), Color.BLACK)
97
98      # Dibujo del espacio estrellado
99      for star in stars:
100         draw_circle(star, 1.0, Color.WHITE)
```

```
101
102     if game_state == 0:
103         # 2. Dibujo de la nave (traslación y rotación)
104         draw_set_transform(Vector2(ship_x, ship_y), angle, Vector2.ONE)
105
106         if thrust_pressed:
107             # Color naranja para el fuego
108             var colors = PackedColorArray([Color(1.0, 0.59, 0.0)])
109             var points = PackedVector2Array([Vector2(-6, 10), Vector2(6, 10), Vector2(randf_range(-4, 4), randf_range(20, 30))])
110             draw_polygon(points, colors)
111
112         # Cuerpo principal (gris 180)
113         draw_rect(Rect2(-10, -5, 20, 15), Color(0.7, 0.7, 0.7))
114
115         # Cabina superior
116         draw_circle(Vector2(0, -5), 8, Color(0.39, 0.78, 1.0))
117
118         # Patas
119         draw_line(Vector2(-10, 10), Vector2(-15, 15), Color.WHITE, 2.0)
120         draw_line(Vector2(10, 10), Vector2(15, 15), Color.WHITE, 2.0)
121
```

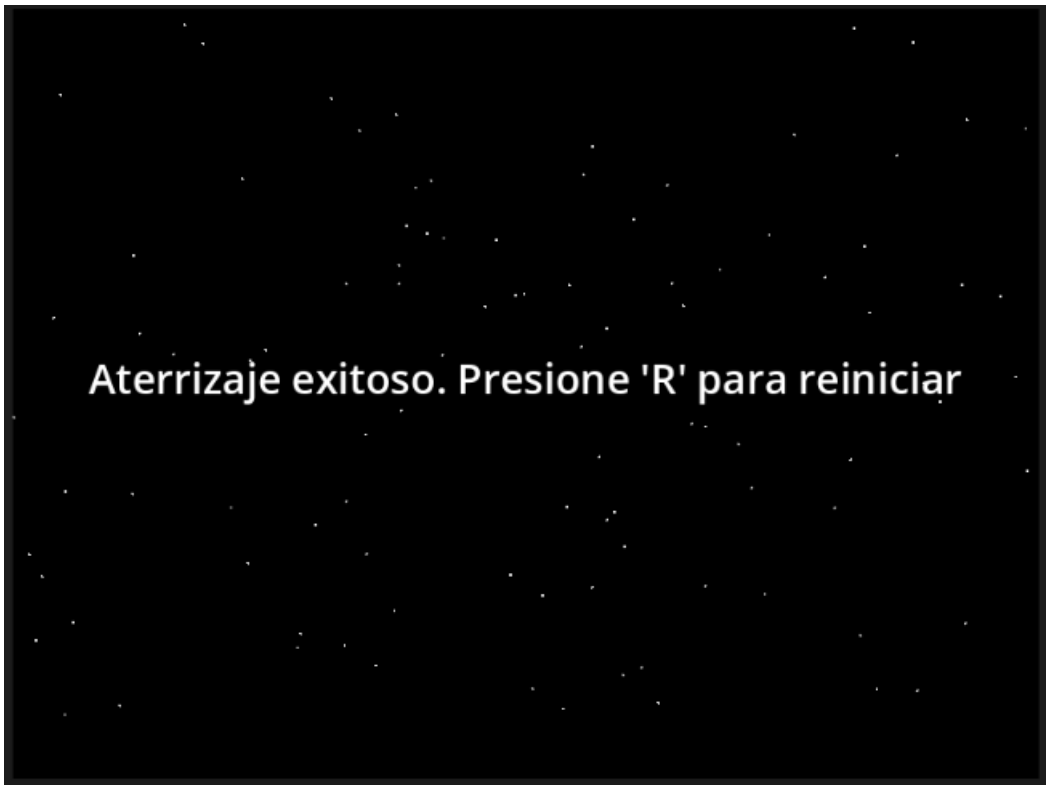
```
122         # Restaura la transformación para los elementos fijos
123         draw_set_transform(Vector2.ZERO, 0.0, Vector2.ONE)
124
125         # 3. Dibujo del terreno y plataforma
126         if textura_tierra != null:
127             draw_texture_rect(textura_tierra, Rect2(0, ground_y, 800, 600 - ground_y), false)
128         else:
129             # Color base del terreno si no carga la textura (gris oscuro 50)
130             draw_rect(Rect2(0, ground_y, 800, 600 - ground_y), Color(0.2, 0.2, 0.2))
131
132         # Plataforma en verde
133         draw_line(Vector2(pad_x, ground_y), Vector2(pad_x + pad_width, ground_y), Color.GREEN, 4.0)
134
135         # 5. Interfaz de telemetría (HUD)
136         var font = ThemeDB.fallback_font
137         draw_string(font, Vector2(30, 40), "ÁNGULO: " + str(snapped(rad_to_deg(angle), 0.01)) + "°", HORIZONTAL_ALIGNMENT_LEFT, -1, 16)
138         draw_string(font, Vector2(30, 65), "Angulo maximo permitido = +/- 30°", HORIZONTAL_ALIGNMENT_LEFT, -1, 16)
139         draw_string(font, Vector2(30, 90), "VELOCIDAD: " + str(snapped(vy*10, 0.01)), HORIZONTAL_ALIGNMENT_LEFT, -1, 16)
140         draw_string(font, Vector2(30, 115), "Velocidad máxima permitida = 15", HORIZONTAL_ALIGNMENT_LEFT, -1, 16)
141
142         var altitud_real = max(0, ground_y - (ship_y + 15))
```

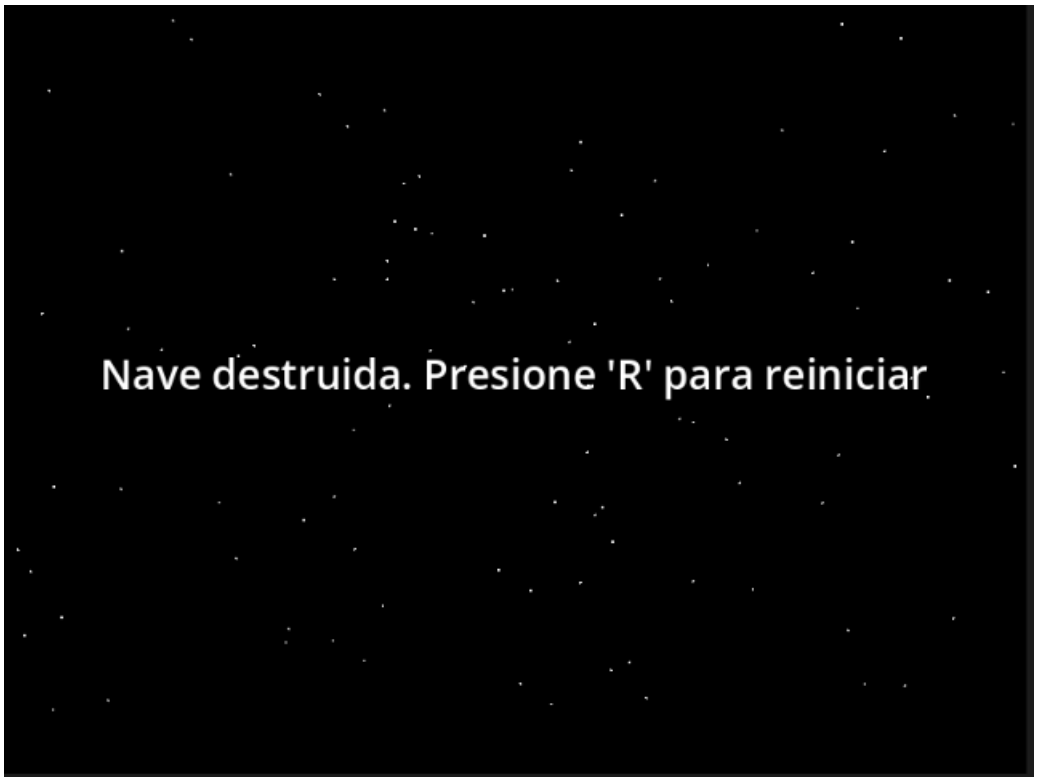
```
143             draw_string(font, Vector2(0, 40), "ALTITUD: " + str(snapped(altitud_real, 0.01)), HORIZONTAL_ALIGNMENT_RIGHT, 770, 16)
144
145         elif game_state == 1:
146             var font = ThemeDB.fallback_font
147             draw_string(font, Vector2(0, 300), "Aterrizaje exitoso. Presione 'R' para reiniciar", HORIZONTAL_ALIGNMENT_CENTER, 800, 32)
148         elif game_state == 2:
149             var font = ThemeDB.fallback_font
150             draw_string(font, Vector2(0, 300), "Nave destruida. Presione 'R' para reiniciar", HORIZONTAL_ALIGNMENT_CENTER, 800, 32)
151         elif game_state == 3:
152             var font = ThemeDB.fallback_font
153             draw_string(font, Vector2(0, 300), "Presione 'S' para comenzar el simulador", HORIZONTAL_ALIGNMENT_CENTER, 800, 32)
154
155 func reiniciar_juego():
156     ship_x = randf_range(50, 800 - 50)
157     ship_y = 50.0
158     vx = 0.0
159     vy = 0.0
160     angle = PI / 2.0 # Equivalente a HALF_PI
161     pad_x = randf_range(0, 800 - pad_width)
162     game_state = 0
163
```

Resultado:




ÁNGULO: -18.86°
Angulo maximo permitido = +/- 30°
VELOCIDAD: 2.67
Velocidad máxima permitida = 15
ALTITUD: 264.01


Aterrizaje exitoso. Presione 'R' para reiniciar


Nave destruida. Presione 'R' para reiniciar